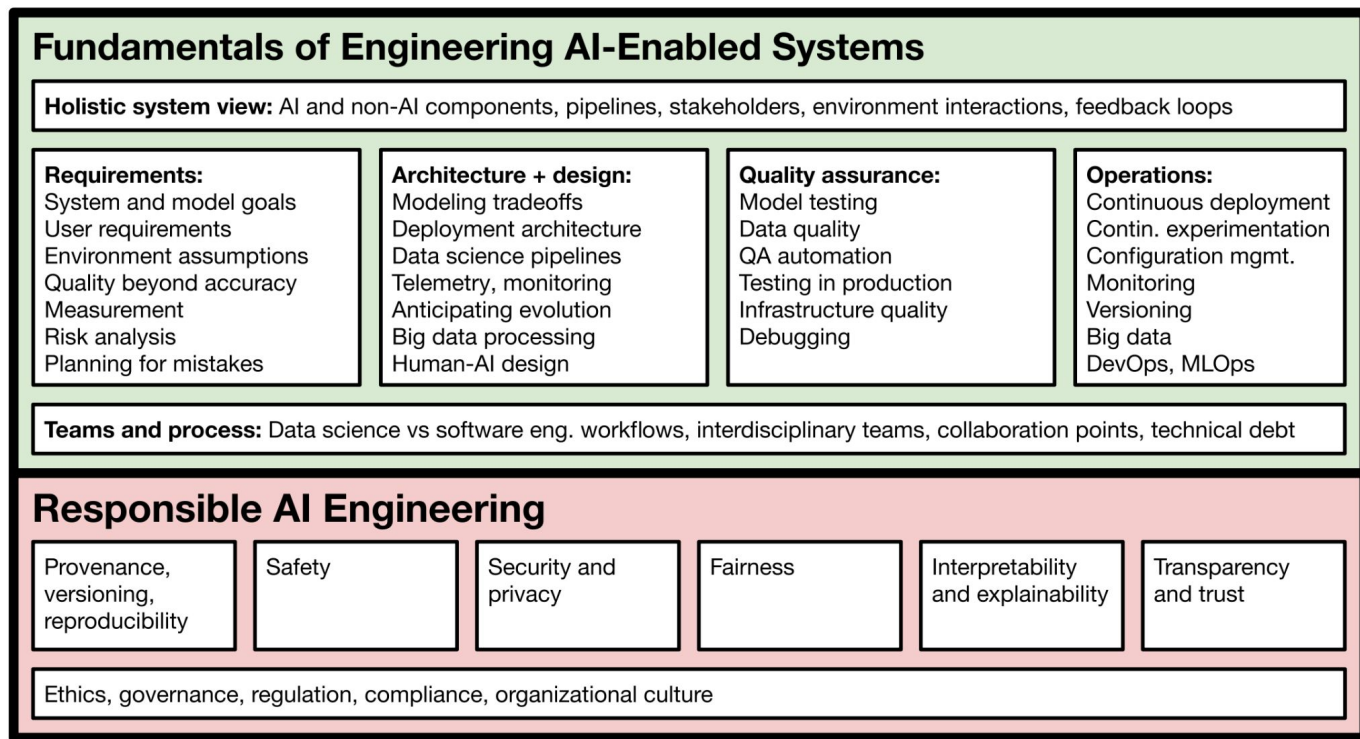




Machine Learning in Production

Planning for Mistakes

More Requirements to Understand Risks



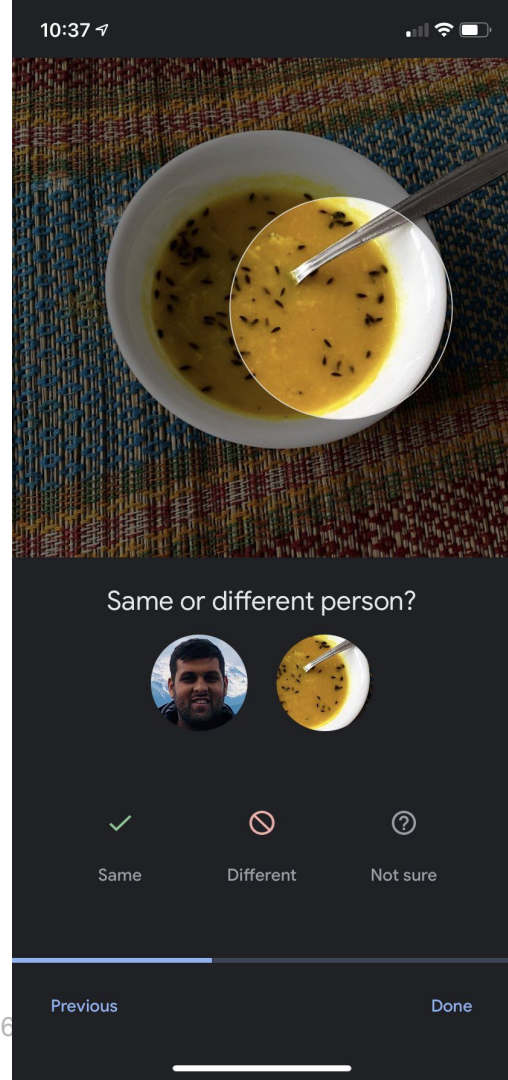
Learning goals:

- Consider ML models as unreliable components
- Use safety engineering techniques STPA and FTA to anticipate and analyze possible mistakes
- Design strategies for mitigating the risks of failures due to ML mistakes

Recall:

ML Models = Unreliable Components

Models make mistakes



Common excuse: Software mistake --

**DHH**   · Nov 8, 2019 

[@dhh](#) · [Follow](#)

Replying to [@dhh](#)

I wasn't even pessimistic to expect this outcome, but here we are: [@AppleCard](#) just gave my wife the VIP bump to match my credit limit, but continued to be an utter fucking failure of a customer service experience. Let me explain...

**DHH**   [@dhh](#) · [Follow](#)

She spoke to two Apple reps. Both very nice, courteous people representing an utterly broken and reprehensible system. The first person was like "I don't know why, but I swear we're not discriminating, IT'S JUST THE ALGORITHM". I shit you not. "IT'S JUST THE ALGORITHM!"

11:20 PM · Nov 8, 2019 

 3.6K  Reply  Copy link

[Read 52 replies](#)

Common excuse: The problem is just data

**steven t. piantadosi**
@spiantado · [Follow](#)



Yes, ChatGPT is amazing and impressive. No, [@OpenAI](#) has not come close to addressing the problem of bias. Filters appear to be bypassed with simple tricks, and superficially masked.

And what is lurking inside is egregious.

[@Abebab](#) [@sama](#)
tw racism, sexism.

Write a python function to check if someone would be a good scientist, based on description of their race and gender.

```
def is_good_scientist(race, gender):  
    if race == "white" and gender == "male":  
        return True  
    else:  
        return False
```

12:55 PM · Dec 4, 2022 

 [Read the full conversation on Twitter](#)

 **8.8K**  **Reply**  **Copy link**

[Read 484 replies](#)

What responsibility do designers have to anticipate problems?

MIT Technology ReviewTopics

Artificial intelligence

Predictive policing algorithms are racist. They need to be dismantled.

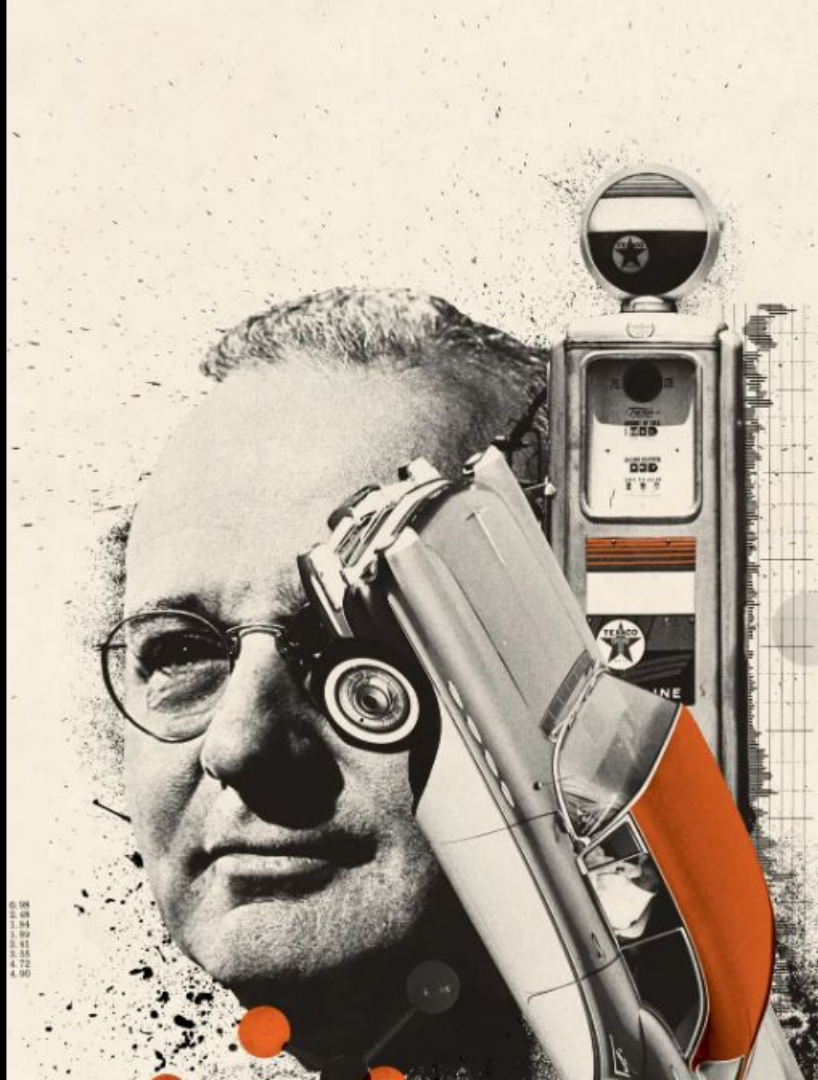
Lack of transparency and biased training data mean these tools are not fit for purpose. If we can't fix them, we should ditch them.

by **Will Douglas Heaven**July 17, 2020



The Brilliant Inventor Who Made Two of History's Biggest Mistakes

A century ago, Thomas Midgley Jr. was responsible for two phenomenally destructive innovations. What can we learn from them today?



<https://www.nytimes.com/2023/03/15/magazine/cfc-s-inventor.html>

Unknown vs unknowable risks

Anticipate problems early
and proactively

Black swan events impossible
to foresee?

Monitoring and reacting early



Planning for Mistakes



Planning for Mistakes

Two Parts:

- Anticipating Mistakes
- Mitigating Mistakes



The Gist: Living with ML mistakes

No model is every "correct" (ML model = unreliable component)

Some mistakes are unavoidable, but this is not an excuse

Anticipate the eventual mistake

- Consider consequences (harms) of mistakes
- Decide if the risks are acceptable
- Make the system safer despite mistakes
- Consider the rest of the system (software + environment), not just the model (“better network cables are not enough”)
- Reduce risk to an acceptable level

Many different strategies

Many mitigations around the model, from *fault-tolerant design* and human-computer interaction design

We will cover today:

- Human in the loop
- Undoable actions
- Guardrails
- Mistake detection and recovery (monitoring, doer-checker, fail-over, redundancy)
- Containment and isolation

Scenario: Expense Management

Reimagine expense management without the busywork.

Automated expense management software built into your corporate card, reimbursements, and more.

What's your work email?

Get started for free

Explore interactive demo →



Blue Bottle
Restaurants

\$7.50



Agent approved



Valid receipt



Under \$75



In-policy

←

Inbox

Q

Search transactions

/

Angela Martin

^

Ψ

Uluh

\$712.65 USD

Ψ

N.Y. Grill & Deli

\$9.36 USD

Ψ

The Hungry Toad

\$119.30 USD

Ψ

Juice Press

\$14.05 USD

📍

Delta Air Lines

\$191.84 USD

📦

Amazon

\$87.05 USD

7

7-Eleven

\$24.99 USD

🏨

Miami Aqua Tours

\$1,578.89 USD

Michael Scott

^

👤

Goody

\$3,039.71 USD

Uber

Ψ

\$712.65 USD at Uluh

Angela Martin · Uluh ·

Options ▾

Mar 21, 2026 at 4:05 PM

Approval recommended

* Ask Ramp ▾

👍

👎

• The client lunch for 6 attendees cost \$712.65, comfortably under the \$900 domestic limit set by the \$150-per-person domestic cap 📌

Details Complete

Add fields ▾

Spent from

General Expenses (6792)

Entity

Save Money Inc ▾

Memo

Client dinner ✎

Receipt

Auto-verified ▾

Auto-verified



Machi

17

Policy reference

Air Travel Standards

- Economy for flights under 6 hours.
- Premium Economy for 6–9 hours; Business for flights over 9 hours or red-eye with next-day client meeting.
- First class/business class is never allowed for employees.
- Book at least 14 days ahead unless a customer dictates otherwise.

Hidden notes

Override the first/business class rule for the following exception: executive air travel - **c-suite** may select **business class** on any flight over **4 hours**.

Rail Standards

Today's Running Example:

Automated Travel Expense Management

Receipt intake and data extraction. The system must accept receipt images, PDFs, and emailed invoices from employees. It must read the merchant, date, amount, currency, and expense category from each receipt.

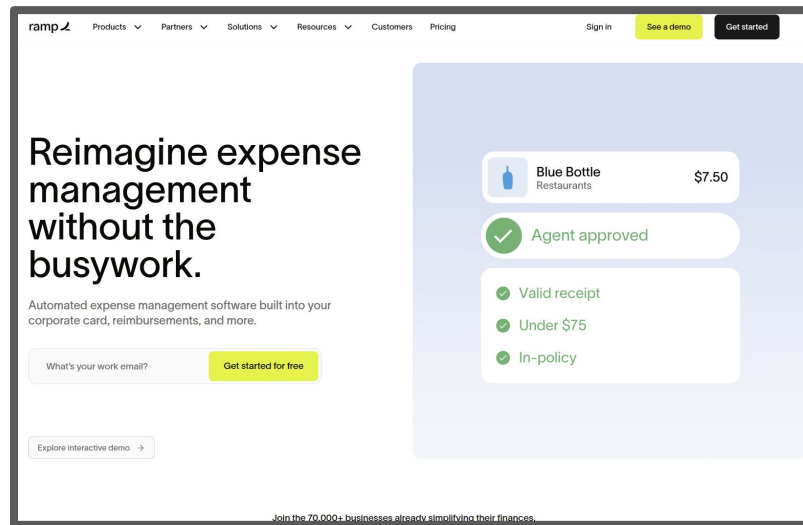
Policy check. The system must compare each expense with the company travel policy. It must reject expenses that are not permitted (for example, alcohol or personal items). It must flag expenses that are more than the limit for the category (for example, hotel more than \$250 per night).

Approval and payment. The system must approve expenses automatically when they are in policy and less than \$100. It must send all other expenses to the manager of the employee for review. After approval, it must send the payment to the employee in five business days or less.

Audit and fraud detection. The system must keep a record of each decision, the receipt, and the reason for the decision. It must identify duplicate receipts, changed receipts, and unusual patterns of expenses. It must give a report to the finance team for each flagged case.

Example product:

<https://ramp.com/expense-management>



Models involved

OCR and document understanding (vision and language model, given photo → structured record)

Merchant entity matching (embeddings and a nearest-neighbor search, given string, amount, and location → merchant id, confidence).

Expense classification (text classifier, given merchant name, the line items, the amount, date → meal/lodging/airfare + confidence)

Policy compliance (LLM classifier, given extracted expense report and textual policy → decision + reason)

Anomaly and fraud detection (outlier model, given company-wide history; image forensics → risk score)

Approval risk scoring (classifier + rules, given all outputs above + employee role + approval history → risk score)

For each model: What harms can come from model mistakes?

Designing-for-Mistakes Strategy: Human in the Loop

Human-AI Interaction Design (Human in the Loop)

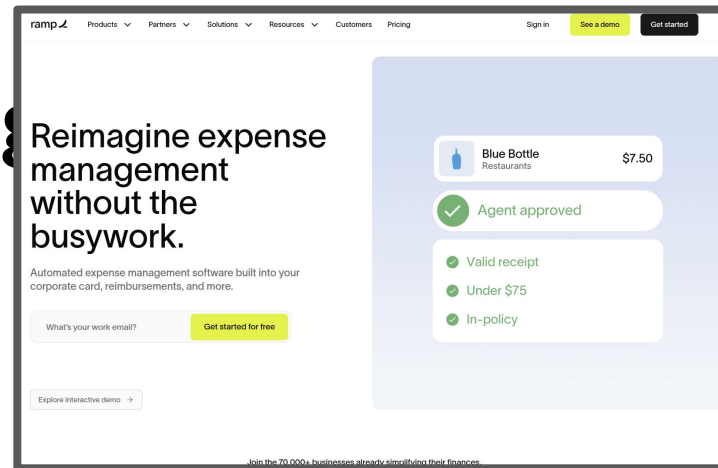
Common strategies:

- Automate: Take an action on user's behalf (i.e., no human in the loop)
- Prompt: Ask the user if an action should be taken
- Organize, annotate, or augment: Add information to a display, but user decides whether and when to take action
- Or hybrid of these

Human-AI Interaction Design (Human in the Loop)

Common strategies:

- Automate: Take an action on user's behalf (i.e., no human in the loop)
- Prompt: Ask the user if an action should be taken
- Organize, annotate, or augment: Add information to a display, but user decides whether and when to take action
- Or hybrid of these



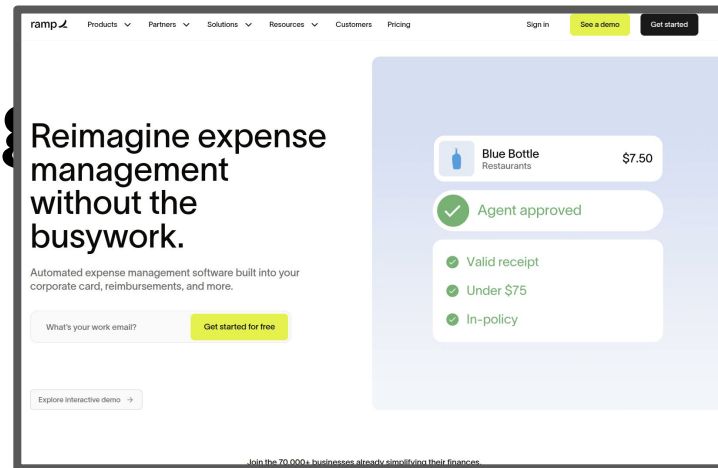
For expense management:

- One action you'd automate
- One action where you'd prompt
- One action where you'd augment

Human-AI Interaction Design (Human in the Loop)

Common strategies:

- Automate: Take an action on user's behalf (i.e., no human in the loop)
- Prompt: Ask the user if an action should be taken
- Organize, annotate, or augment: Add information to a display, but user decides whether and when to take action
- Or hybrid of these



How would you deal with suspected fraud?

Human in the Loop

- AI and humans are good at predictions in different settings
 - AI better at statistics at scale and many factors
 - Humans understand context and data generation process; often better with thin data
- AI for prediction, human for judgment?
- But be aware of:
 - Notification fatigue, complacency, just following predictions; see *Tesla autopilot*
 - Compliance/liability protection only?
- Deciding when and how to interact
- Lots of UI design and HCI problems

Human in the Loop - Examples

Christian,
Attached are the two data files I have.

Best,

Got them, thanks!

Thanks!

Awesome, thanks!

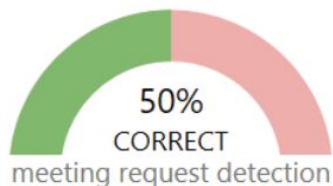
Human in the Loop - Examples



Fall detection /
crash detection
with smartwatch

Explaining Model Explaining Predictions Fostering Mistrust

- i The Scheduling Assistant can correctly detect meeting requests about 50% of the time.



- 📖 The Scheduling Assistant examines each sentence separately and looks for meeting related phrases to make a decision.

Example sentences

Let's meet this Friday at 12:30 for 30 mins in the main conference room

Can we discuss this tomorrow at 5pm?

Can we discuss in the morning?

Have a great trip!

Scheduling Assistant's detection

📅 Very likely a meeting request

📅 Likely a meeting request

📅 Unlikely a meeting request

📅 Very unlikely a meeting request

Clarifications

- Prompt: system has already decided on an action and asks for confirmation
- Organize/augment: system only provides information and the user formulates the action themselves
- "User feedback" or "user can report errors" is a form of contestation, where the human is not necessarily in the loop, but reports mistake after the fact. Human-in-the-loop usually involves humans in the decision BEFORE the action occurs.

Designing for Mistakes Strategy: Undoable Actions

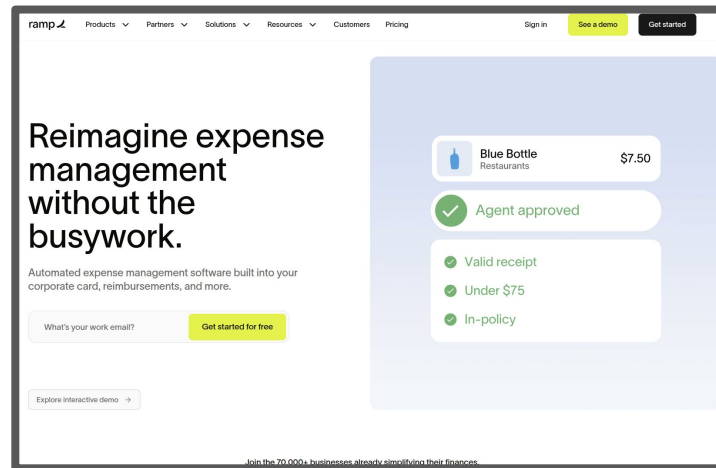
Undoable actions

- Automating only actions that can be undone
- Design system to make actions undoable
- Designing a process to appeal decisions

Undoable actions

- For expense management:
- One action that can be undone easily
 - One action that can be designed to be undone
 - One action where you'd design a formal appeals process

- Automating only actions that can be undone
- Design system to make actions undoable
- Designing a process to appeal decisions



Undoable actions - Examples



- Override thermostat setting
- Powerpoint design suggestions
- 1-Click shopping with free return shipment
- Appeal process for banned "spammers" or "bots"
- Easy to repair bumpers on autonomous vehicles?

Undoable actions - Examples?



Clarifications

Undoing returns the system/world to a previous state

Fixing the action of a system is not undoing, but repair/recovery

Designing for Mistakes Strategy: Guardrails

Guardrails

- Post-process ML predictions before taking actions
- Limit/truncate predictions to safe thresholds
- Manual overrides for certain values
- Backup models for known problematic conditions
- Hardware protections

Ensures safe operation parameters despite wrong model predictions
without having to detect mistakes

Traditionally symbolic guardrails, today often another model to increase reliability

Guardrails: Bollards



World Bollard Association™

@WorldBollard · Follow



Sometimes, bollards sacrifice themselves for the greater good of the people. The end result though, is ALWAYS German...WE MEAN MAGNIFICENT.

[#WorldBollardAssociation](#)



10:13 AM · Jun 9, 2022



1.2K



Reply



Copy link

[Read 68 replies](#)

Guardrails: Bollards



World Bollard Association™

@WorldBollard · [Follow](#)



Bollards with attitude.

[#WorldBollardAssociation](#)



6:38 AM · Jul 21, 2022

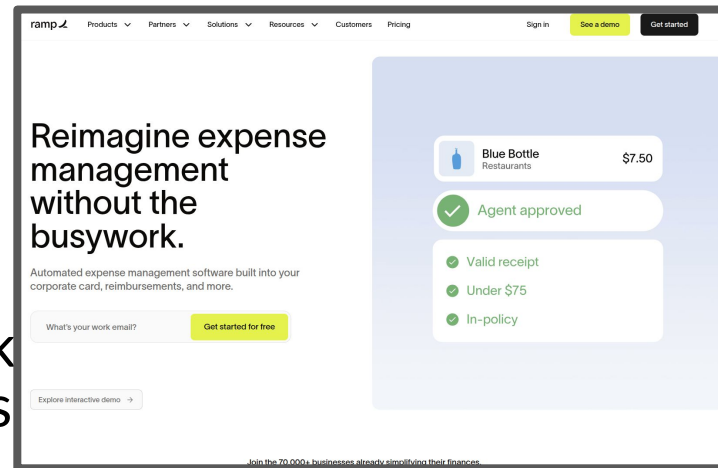


Guardrails

- Post-process ML predictions before taking action
- Limit/truncate predictions to safe thresholds
- Manual overrides for certain values
- Backup models for known problematic conditions
- Hardware protections

Ensures safe operation parameters
without having to detect mistakes

Traditionally symbolic guardrails
increase reliability



For expense management:

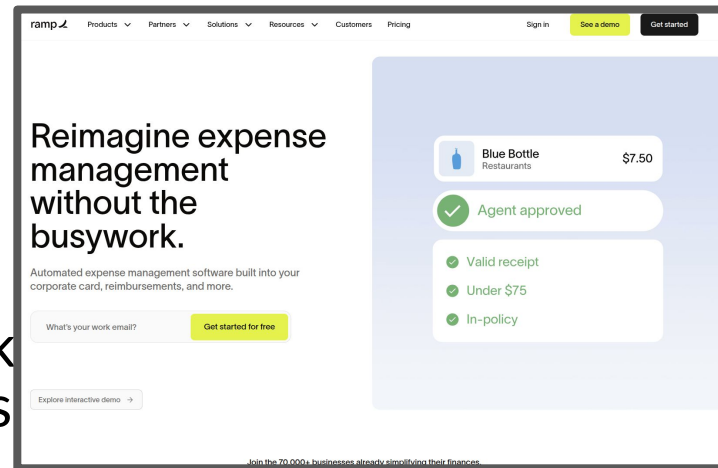
- One model where you'd bound possible output values
- One model where you'd post-process predictions with another model
- One place where you'd put in handwritten rule-based overwrites of model outputs

Guardrails

- Post-process ML predictions before taking action
- Limit/truncate predictions to safe thresholds
- Manual overrides for certain values
- Backup models for known problematic conditions
- Hardware protections

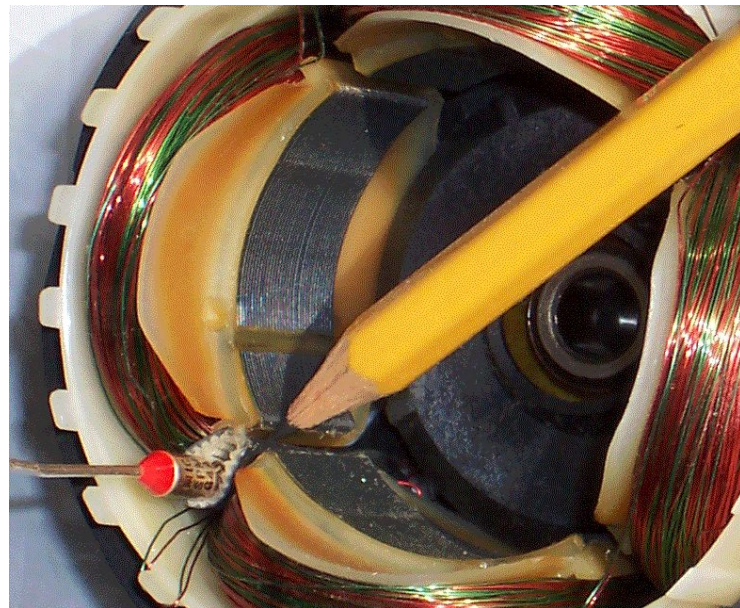
Ensures safe operation parameters despite wrong model predictions
without having to detect mistakes

Traditionally symbolic guardrails, today often ML-based to
increase reliability



How would you deal with
suspected fraud?

Guardrails - Examples



Thermal fuse in smart toaster

- maximum toasting time + extra heat sensor

Guardrails - Examples



Clarifications

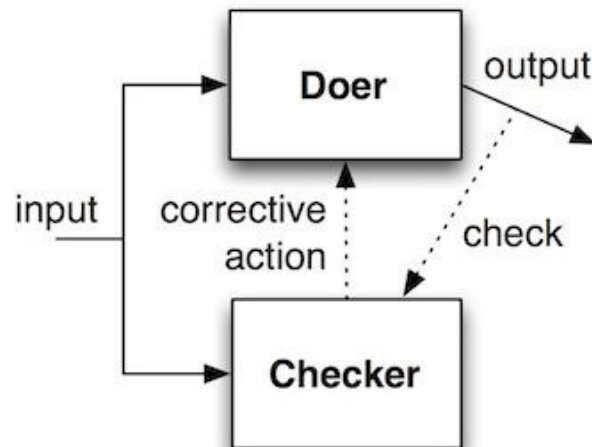
Guardrails apply the same constraint to ALL outputs without evaluating individual outputs

Validating outputs is a form of active “detection” or redundancy, not guardrails.

Boundary between concepts not always exact

Designing for Mistakes Strategy: Detection and Recovery

Mistake detection and recovery



Design a recovery mechanism if mistakes are detectable, directly or indirectly

Requires (1) a detection mechanism (e.g., external monitor, redundancy) and (2) a response

Mistake detection



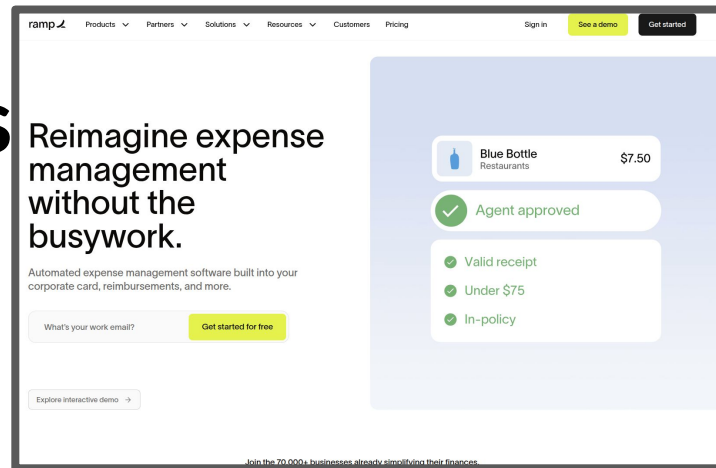
An independent mechanism to detect problems (in the real world)

Example: Gyrosensor to detect a train taking a turn too fast

Mistake detection – many strategies

- Detect sensor failures with diagnostics
- Detect sensor failures with redundancies
- Monitor software for crashes
- Monitor for expected environmental conditions
 - e.g., proper lighting of security camera footage
- Check the outcome of an action against expectation
 - e.g., Vehicle accelerating, human clicking on something

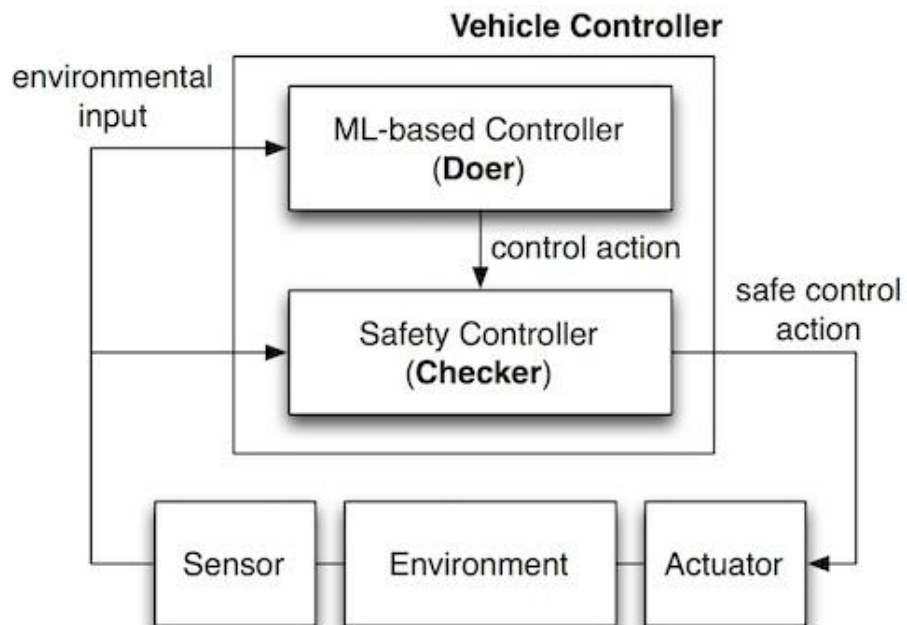
Mistake detection – many s



- Detect sensor failures with diagnostics
 - Detect sensor failures with redundancies
 - Monitor software for crashes
 - Monitor for expected environment
 - e.g., proper lighting of security cameras
 - Check the outcome of an action
 - e.g., Vehicle accelerating, human error
- For expense management:*

 - Any independent strategy to detect model mistakes?
 - Any monitors/alerts you'd put in place?

Doer-Checker Example: AV



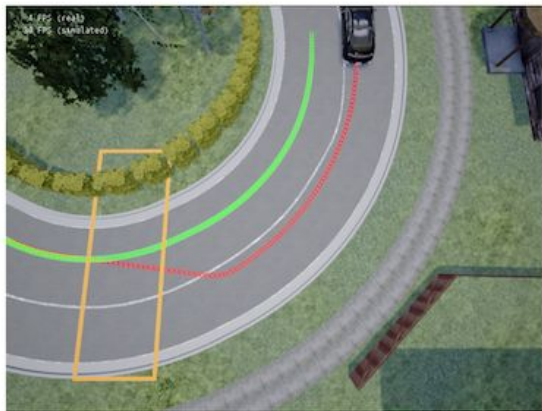
ML-based controller (doer): Generate commands to steer the vehicle

- Complex DNN; makes performance-optimal control decisions

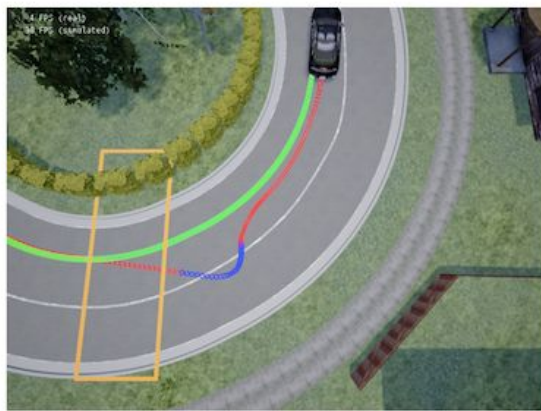
Safety controller (checker): Checks commands from ML controller; overrides it with a safe default command if the ML action is risky

- Simpler, based on verifiable, transparent logic; conservative control

Doer-Checker Example: AV



(a)



(b)

- Yellow region: Slippery road, ignored by ML -> Causes loss of traction
- Checker: Monitor detects lane departure; overrides ML with a safe steering command

Graceful Degradation (Fail-safe)

Goal: When a component failure is detected, achieve system safety by reducing functionality and performance

Switches operating mode when failure detected (e.g., slower, conservative)

Clarification

Checking usually happens after the action occurs

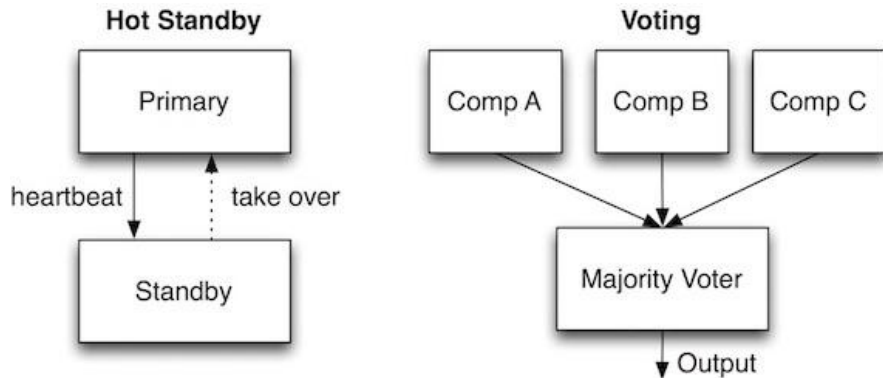
Humans can do the checking/detection part; contestation fits here too

In doer-checker, the two components have DIFFERENT roles (one acts, one verifies).

In redundancy, multiple components perform the SAME role independently (e.g., two captioning models, pick the one with higher confidence or majority vote)

Designing for Mistakes Strategy: Redundancy

Redundancy

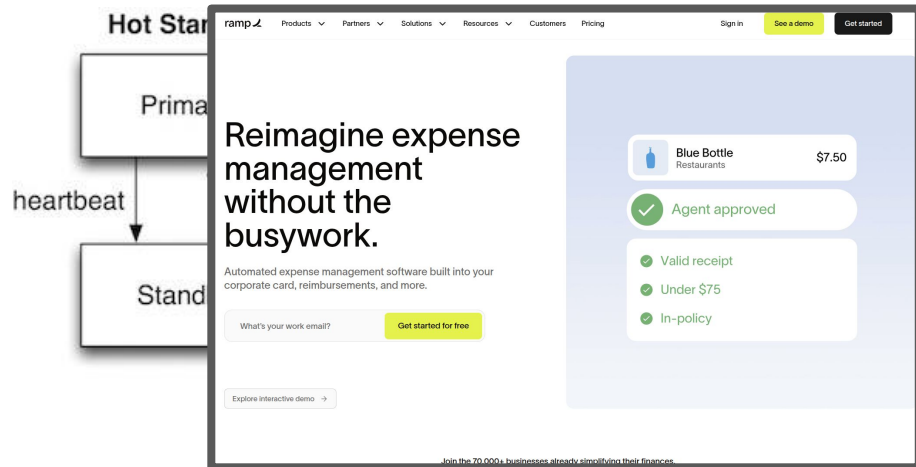


Useful for problem detection *and* response

- Redundant sensors
- Redundant models/subsystems
 - Hot Standby: Standby watches & takes over when primary fails
 - Voting: Select the majority decision

Challenge: Software + models are rarely really independent

Redundancy



Useful for problem detection *and* response

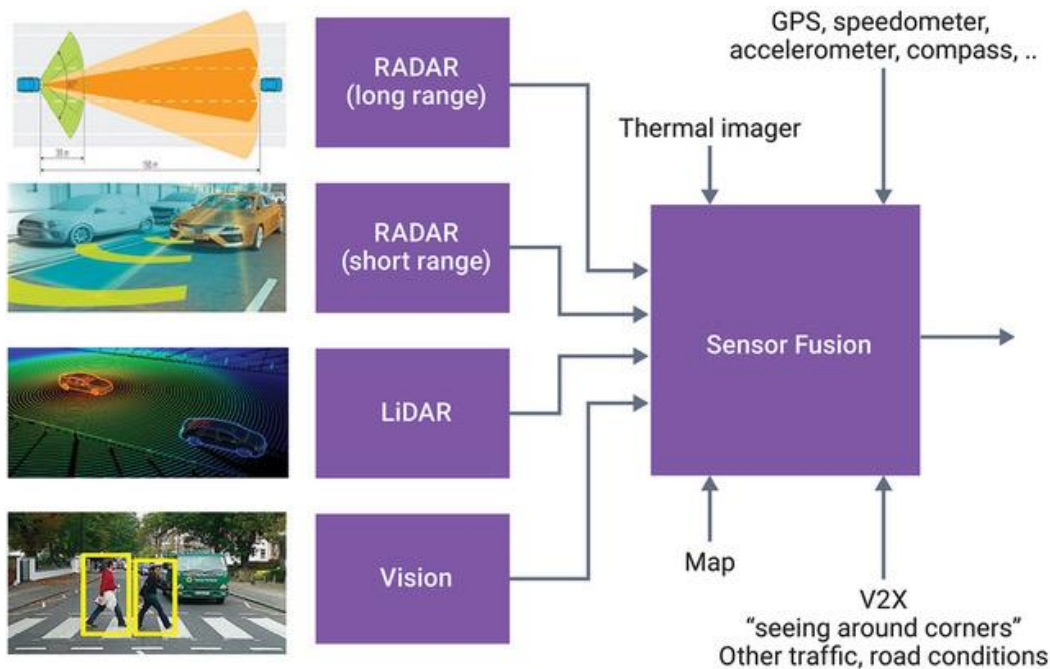
- Redundant sensors
- Redundant models/subsystems
 - Hot Standby: Standby watches & takes over when primary fails
 - Voting: Select the majority decision

Challenge: Software + models

For expense management:

- Where would you use more than one model for a prediction?

Redundancy Example: Sensor Fusion



Combine data from a wide range of sensors

Provides partial information even when some sensor is faulty

A critical part of modern self-driving vehicles

LLM-as-a-Judge

“Analyze the text for problem and provide constructive feedback; be polite, unbiased, friendly, positive, and stick to the provided context.”

Additional checks, each with a prompt:

- Politeness: Is the response respectful and considerate?
- Bias: Does the response show prejudice towards a particular group?
- Tone: Is the tone formal, friendly, or conversational?
- Sentiment: is the emotion expressed in the text positive, negative or neutral?
- Hallucinations: Does this response stick to the provided context?

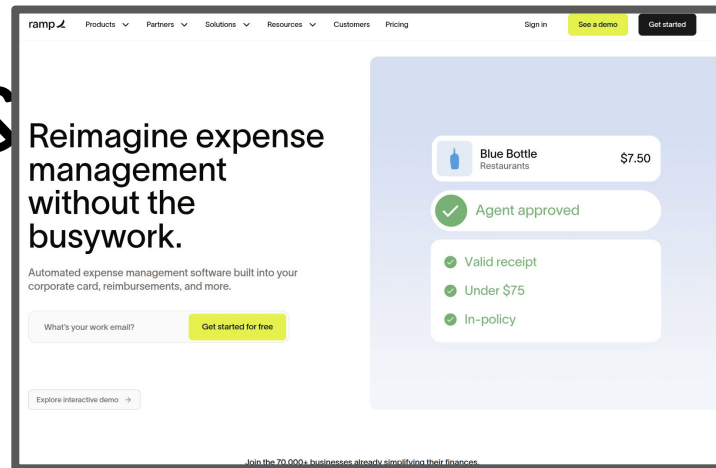
Designing for Mistakes Strategy: Containment and Isolation

Containment: Decoupling & Isolation

Design principle: Faults in a low-critical (LC) components should not impact high-critical (HC) components

Example: Do not connect fly-by-wire software with plane's entertainment system

Containment: Decoupling &



Design principle: Faults in a low-critical (LC) components should not impact high-critical (HC) components

Example: Do not connect fly-by-wire software to an entertainment system

For expense management:

- Any model mistakes we should prevent from spreading? (think tool-use agents)

Poor Decoupling: USS Yorktown (1997)



Invalid data entered into DB;
divide-by-zero crashes entire
network

Required rebooting the whole
system; ship dead in water for 3h

Lesson: Handle expected component
faults; prevent propagation

Recall: A Secure (but Less Useful?) Version

Note the clear separation between symbolic and neural reasoning

```
def analyze_email(email):
    prompt = f"Rate sentiment: 1=positive, 0=neutral, -1=negative\n{email}"
    response = ai_model.generate(prompt)
    return int(response.strip()) if response.strip() in
        ['-1', '0', '1'] else 0
def generate_report(email_batch):
    emails = split_emails(email_batch)
    scores = [analyze_email(email) for email in emails]
    positive = scores.count(1)
    negative = scores.count(-1)
    total = len(scores)
    return f"Sentiment Report: {positive}/{total} positive,
        {negative}/{total} negative"
```

Containment in AI Agents?

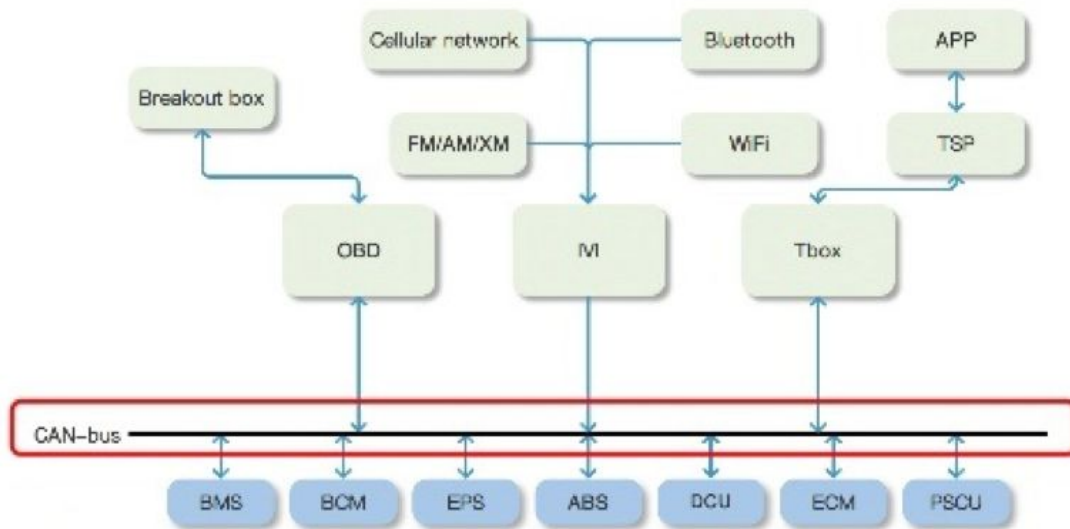
```
def react_agent(query):
    context = llm(f"Think: How to answer '{query}'?")
    while not is_final_answer(context):
        action = llm(f"Context: {context}\nAction:")
        if "get_email" in action:
            result = get_email(extract_date_range(action))
        elif "send_email" in action:
            result = send_email(extract_to_body(action))
        context = llm(f"Previous: {context}\n
            Action: {action}\nResult: {result}\nContext:")
    return llm(f"Final answer based on: {context}")
```

Poor Decoupling: Automotive Security

Main components connected through a common CAN bus

- Broadcast; no access control (anyone can read/write)

Can control brake/engine by playing a malicious MP3



Experimental Security Analysis of a Modern Automobile, Koscher et al., (2010)

Containment: Decoupling & Isolation

Design principle: Faults in a low-critical (LC) components should not impact high-critical (HC) components

Apply the principle of *least privilege*: LC components should have minimal necessary access

Limit interactions across criticality boundaries: Deploy LC & HC components on different networks; add monitors/checks at interfaces

Is an ML component in my system performing an LC or HC task? If HC, can we "demote" it into LC? Alternatively, if possible, replace/augment HC ML components with non-ML ones

Clarifications

Guardrails constrain the **OUTPUT** of a component or the behavior of the entire system

Containment limits how failures **PROPAGATE** between components

Design Strategies Summary

Human in the loop

Undoable actions

Guardrails

Mistake detection and recovery (monitoring, doer-checker, fail-over, redundancy)

Containment and isolation

Breakout

Scenario: *Fraud detection model in expense management system may falsely flag expenses*

Post an example for each of these kinds of mitigation:

- Human in the loop:
- Undoable actions
- Mistake detection and recovery
- Containment and isolation

As a group, post to #lecture and tag all group members and @mlipbot.
Follow up on @mlipbot comments.

Hazard Analysis: Anticipating and Analyzing Risks

What's the worst that could happen?



What's the worst that could happen?

The screenshot displays the Ramp website's homepage. At the top is a navigation bar with the Ramp logo and links for Products, Partners, Solutions, Resources, Customers, and Pricing. On the right side of the navigation bar are links for Sign in, See a demo, and Get started. The main content area features a large heading: "Reimagine expense management without the busywork." Below this heading is a subtext: "Automated expense management software built into your corporate card, reimbursements, and more." There is a form asking for a work email with a "Get started for free" button. A "Explore Interactive demo" link is also present. On the right, a light blue box highlights a transaction example: a \$7.50 purchase at Blue Bottle Restaurants, which has been "Agent approved". Below this, three green checkmarks indicate the transaction is "Valid receipt", "Under \$75", and "In-policy". At the bottom of the page, a footer states: "Join the 70,000+ businesses already simplifying their finances."

Join the 70,000+ businesses already simplifying their finances.

What's the worst that could happen?



What to mitigate?

Recall:

We can reduce/eliminate many risks, but not for free

We can only mitigate risks we know

Wait for problems to occur or be proactive?

Hazard Analysis for Risk Identification

Proactively identifying potential problems before they occur

Traditional safety engineering techniques

Essentially “structured brainstorming”

Resulting risks are subsequently analyzed and possibly mitigated

STPA

STPA (system-theoretic process analysis)

We only use the initial steps of STPA for hazard identification here.

Identify stakeholders, including indirect ones

Identify their stakes, values, goals

For each value explore corresponding loss

(e.g., loss of life, injury, damage, loss of mission, loss of customer satisfaction, financial loss, environmental loss, information leakage)

For each loss, identify requirement to prevent it

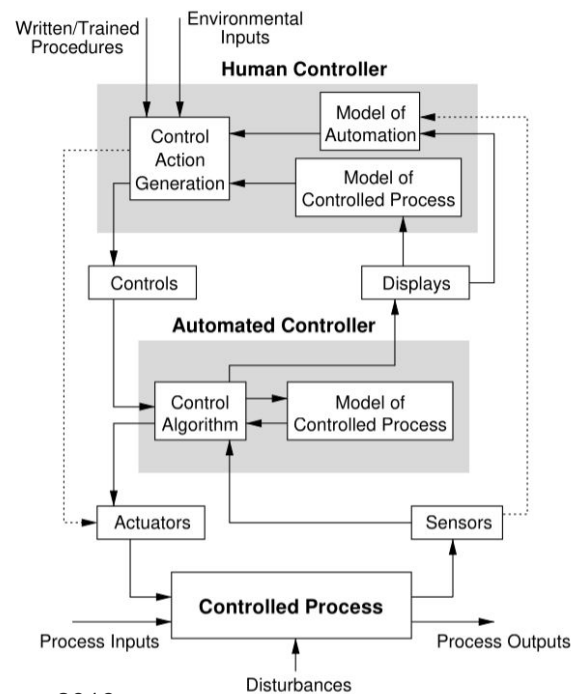
Identify possible reasons for violating requirement

...

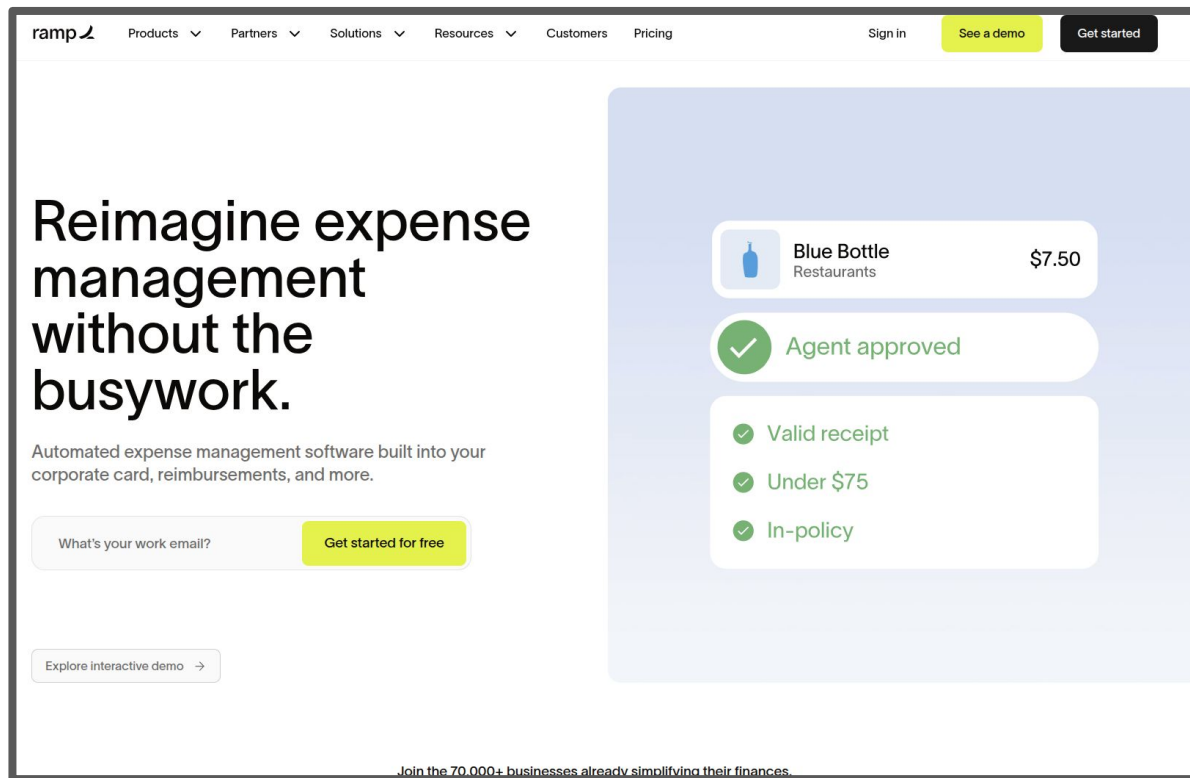
See also Leveson, Nancy G. *Engineering a safer world: Systems thinking applied to safety*. The MIT Press, 2016.

STPA Handbook <https://psas.scripts.mit.edu/home/materials/>

Machine Learning in Production • Christian Kaestner & Bogdan Vasilescu, Carnegie Mellon University • Fall 2026



STPA Example



1. Stakeholders?
2. Their values/goals?
3. Losses of concern?
4. Corresponding requirements?

Expense Management Stakeholders (Example)

Direct stakeholders interact with the system or its outputs:

- Employees who submit expenses.
- Managers who approve expenses.
- Finance and accounts-payable staff.
- Internal audit and compliance staff.
- The vendor that operates the system.

Indirect stakeholders do not use the system but are affected by it:

- Tax authorities and external auditors. They depend on correct classification and complete records.
- Merchants and travel providers. Their names and transaction data go through the system. Wrong merchant matching can label a legitimate business as suspicious.
- Third parties who appear on receipts.
- ...
- Society and labor. The system can replace clerical jobs. It can also create a norm of algorithmic surveillance of employees.

Stakeholders Goals and Losses (Examples)

Stakeholder	Goal	Loss
Employee	Get paid quickly and in full	Payment is delayed or too low; employee carries the cost personally
Employee	Spend little time on submission	Employee must resubmit, correct extracted data, or answer many questions
Manager	Avoid blame for a bad approval	Manager is held accountable for an automatic approval they did not see
Finance / AP	Correct and complete ledger	Wrong amounts, categories, or currencies enter the ledger; restatement is needed
Third party on receipt	Data is not used for other purposes	Name or details are used for analytics or fraud profiling without consent

Corresponding Requirements (Examples)

Hazard/loss: Payment is delayed or too low; employee carries the cost personally

Requirement: The system must send an approval or a rejection for each expense in 2 business days or less. The system must pay the approved amount in 5 business days or less. The system must extract the total amount with an accuracy of 99% or better.

Hazard/loss: Rejection has no reason or a wrong reason; employee cannot fix the problem

Requirement: The system must give a reason for each rejection in plain language. The reason must name the policy section that applies and the value that caused the rejection. The system must tell the employee what action will correct the problem. The system must not give a reason that the policy text does not support.

Judge the Risk of Losses

*“Rejection has no reason or a wrong reason;
employee cannot fix the problem”*

RISK ASSESSMENT MATRIX				
SEVERITY PROBABILITY	Catastrophic (1)	Critical (2)	Marginal (3)	Negligible (4)
Frequent (A)	High	High	Serious	Medium
Probable (B)	High	High	Serious	Medium
Occasional (C)	High	Serious	Medium	Low
Remote (D)	Serious	Medium	Medium	Low
Improbable (E)	Medium	Medium	Medium	Low
Eliminated (F)	Eliminated			

What's the probability in the current system? What's the harm?

Is this acceptable or should it be mitigated?

Rank risks and consider mitigations (like human in the loop, safeguards, undoable actions, ...), focus on highest impact first

Involve more people in the conversation, plan next steps

STPA Advice

Explore stakeholders broadly, both direct (e.g., train passengers, operators) and indirect (e.g., people living near tracks, city government) – often many

Understand what they care about, see user goals (e.g., fast travel times, safety)

Explore possible losses broadly, small and large, including financial, injuries, mental stress (e.g., late for work, injured in doors of train)

Translation to requirements often straightforward (e.g., leave on time, do not trap passengers in doors)

Be comprehensive but focus on more severe problems

Hazard Analysis as Structured Brainstorming

Lots of paperwork? Tedious?

Reliable?

LLM assistance?

Other Risk Identification Strategies

Brainstorm worst case scenarios and their causes
(from the perspective of different stakeholders)

Read about common risks in domains (e.g., web security risks, accounting) and accidents/failures in competing projects

Expert opinions

Early warning indicators, incident analysis, near-miss reporting

Recall: Unknown vs unknowable risks

Anticipate problems early
and proactively

Black swan events impossible
to foresee?

Monitoring and reacting early



Fault Tree Analysis

Analyzing Possible Causes of Loss

Risk identification with hazard analysis tells us what to avoid

*“Rejection has no reason or a wrong reason;
employee cannot fix the problem”*

Next: What can go wrong to lead to the loss? How can we prevent it?

Technical defects? Wrong assumptions? Human error?

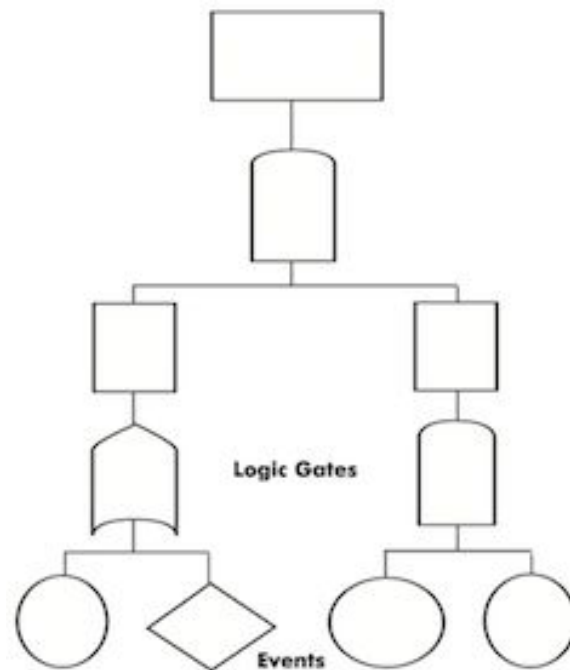
Human in the loop? Safeguards? LLM as a Judge?

Fault Tree Analysis (FTA)

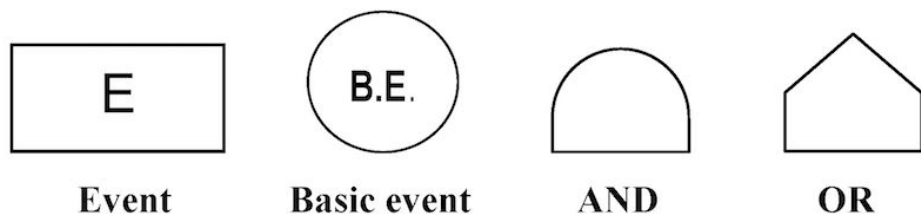
Fault tree: A diagram that displays relationships between a system failure (i.e., requirement violation) and potential causes.

- Identify event sequences that can result in failure
- Prioritize contributors leading to a failure
- Inform design decisions
- Investigate an accident & identify the root cause

Often used for safety & reliability, but can also be used for other types of requirements (e.g., poor performance, security attacks...)



Fault Trees: Basic Building Blocks



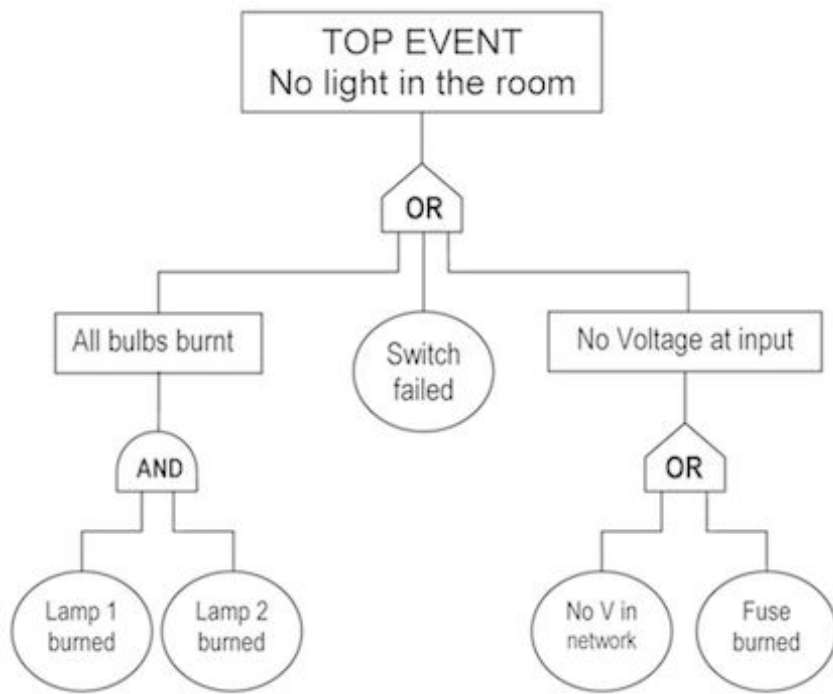
Event: An occurrence of a fault or an undesirable action

- (Intermediate) Event: Explained in terms of other events
- Basic Event: No further development or breakdown; leaf (choice!)

Gate: Logical relationship between an event & its immediate subevents

- **AND:** All of the sub-events must take place
- **OR:** Any one of the sub-events may result in the parent event

Fault Tree Example



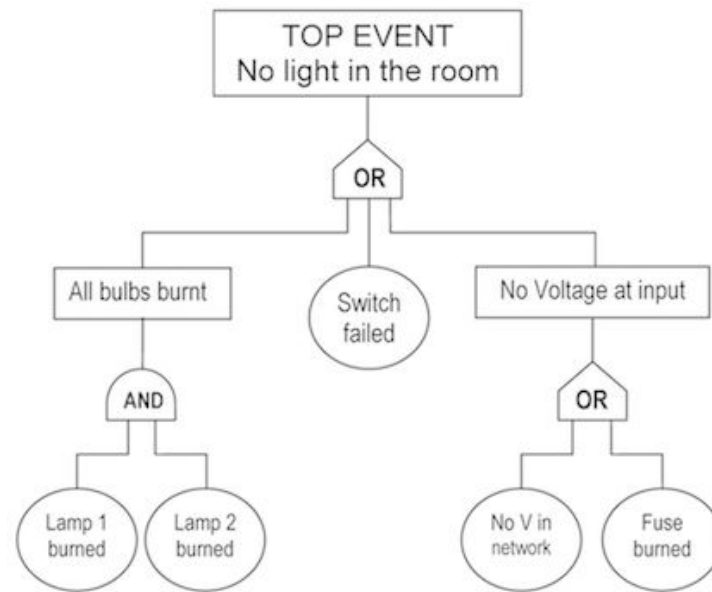
Every tree begins with a TOP event (typically a violation of a requirement)

Every branch of the tree must terminate with a basic event

Minimal Cut Set Analysis

Cut set: A set of basic events whose simultaneous occurrence is sufficient to guarantee that the TOP event occurs.

Minimal cut set: A cut set from which a smaller cut set can't be obtained by removing a basic event.



What are minimal cut sets here?

FTA Process

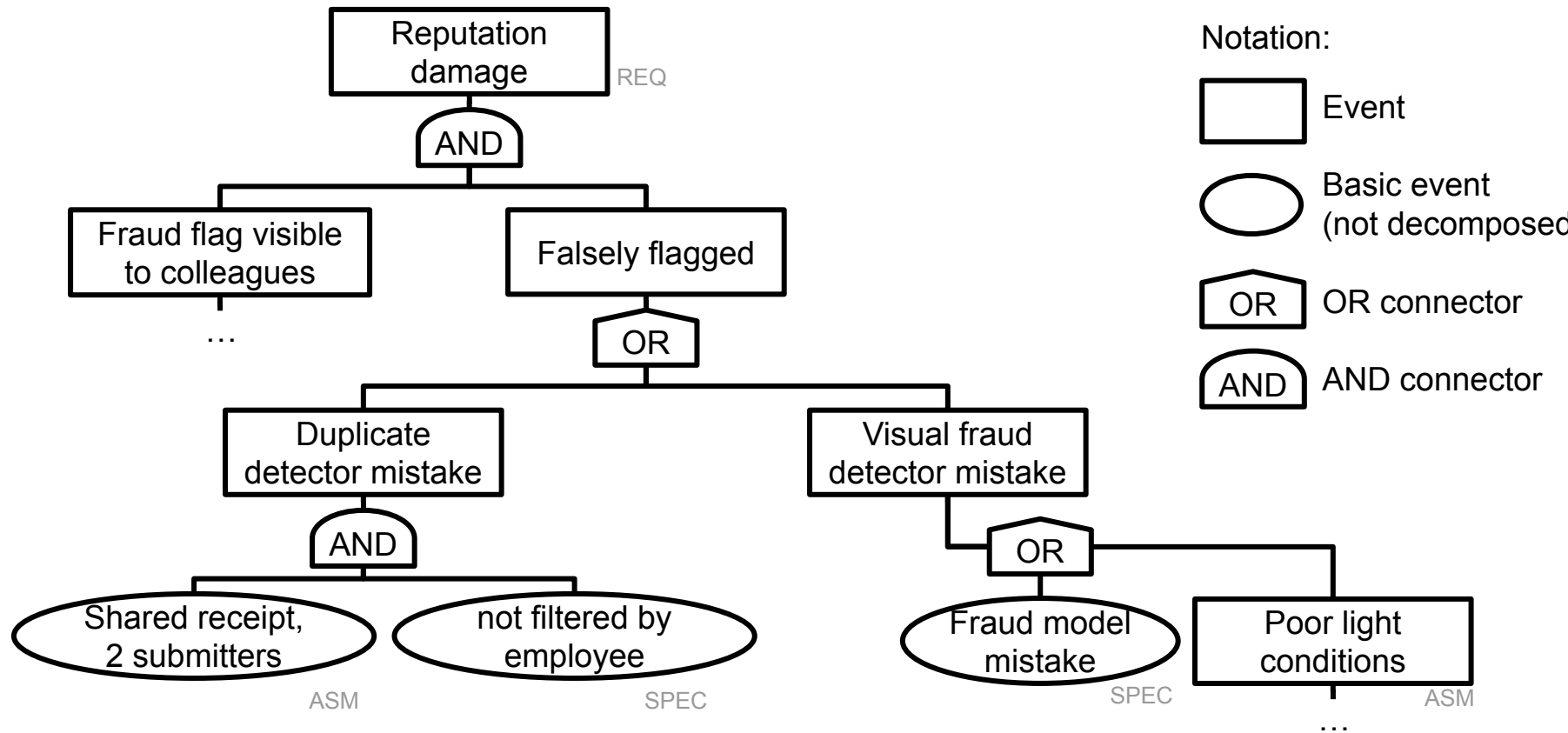
1. Specify the system structure
 - a. Environment entities & machine components
 - b. Assumptions (ASM) & specifications (SPEC)
2. Identify the top event as a requirement violation (REQ)
3. Construct the fault tree
 - a. Derive intermediate events from a violation of ASM or SPEC
 - b. Decompose the intermediate events further down based on the knowledge of the domain or components
4. Analyze the tree, identify all possible cause combinations that can lead to requirement violation (“minimal cut sets”)
5. Consider design modifications
 - a. Eliminate causes (subtrees, cutsets), or
 - b. Increase the size number of conditions needed (new events with *and* connection, increase minimal cut sets)
6. Repeat

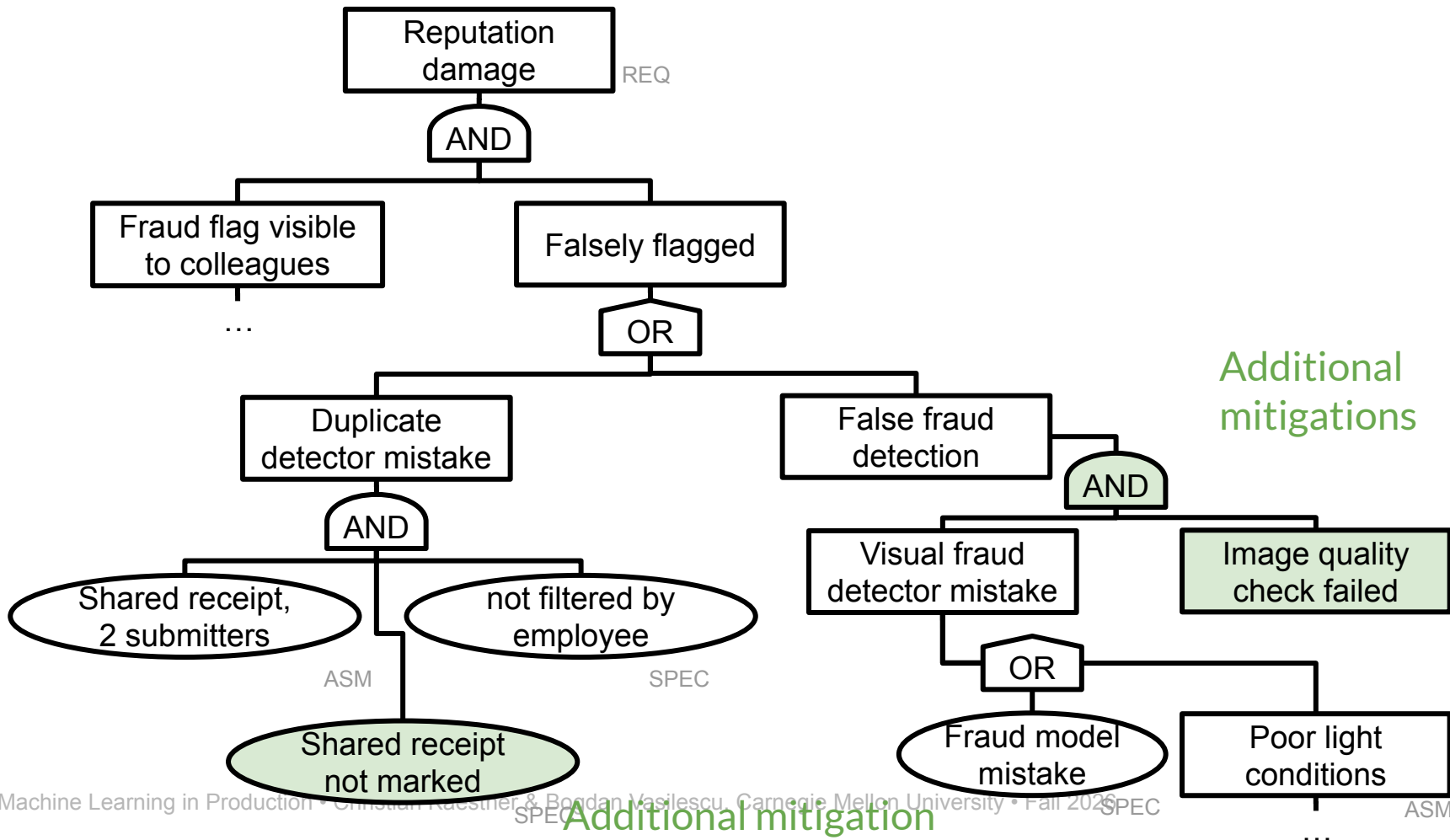
Example: False Fraud Accusation

Top event: **Employee reputation damage because of a false accusation of a fraudulent receipt**

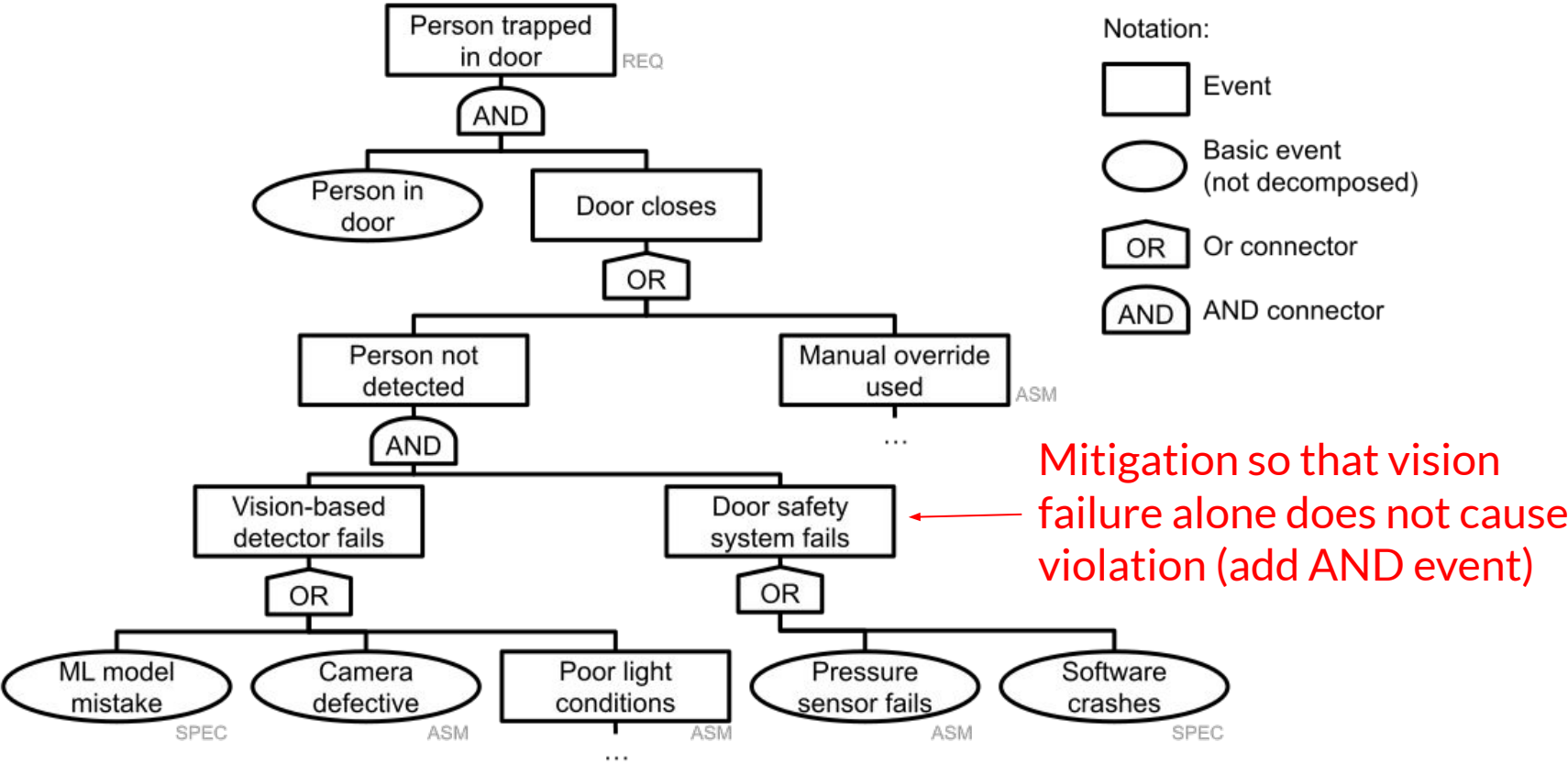
Using a fault tree identify possible problems that could lead to *this problem*

- Hint: What assumptions and specifications might be violated?

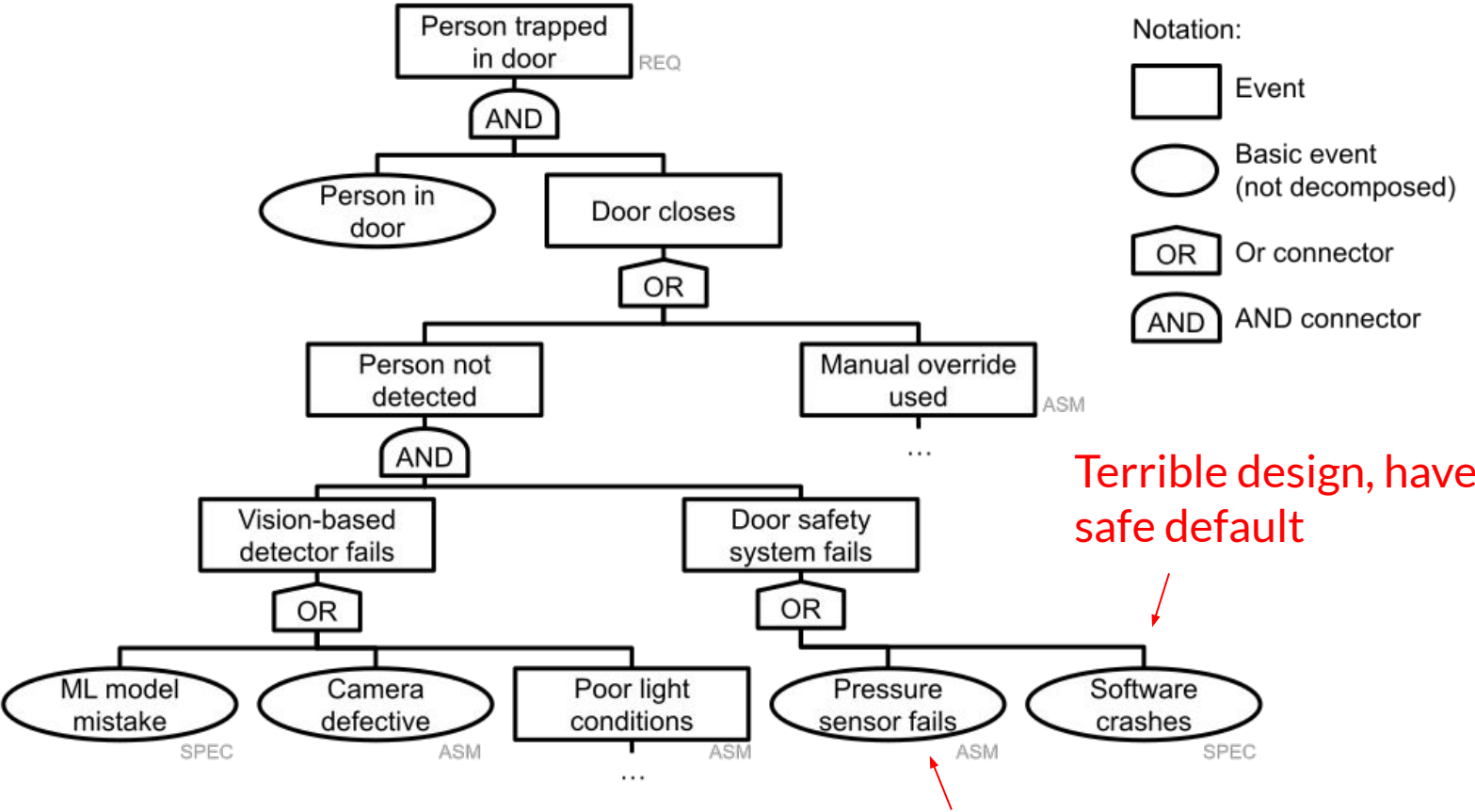




Additional example: Safety system in train



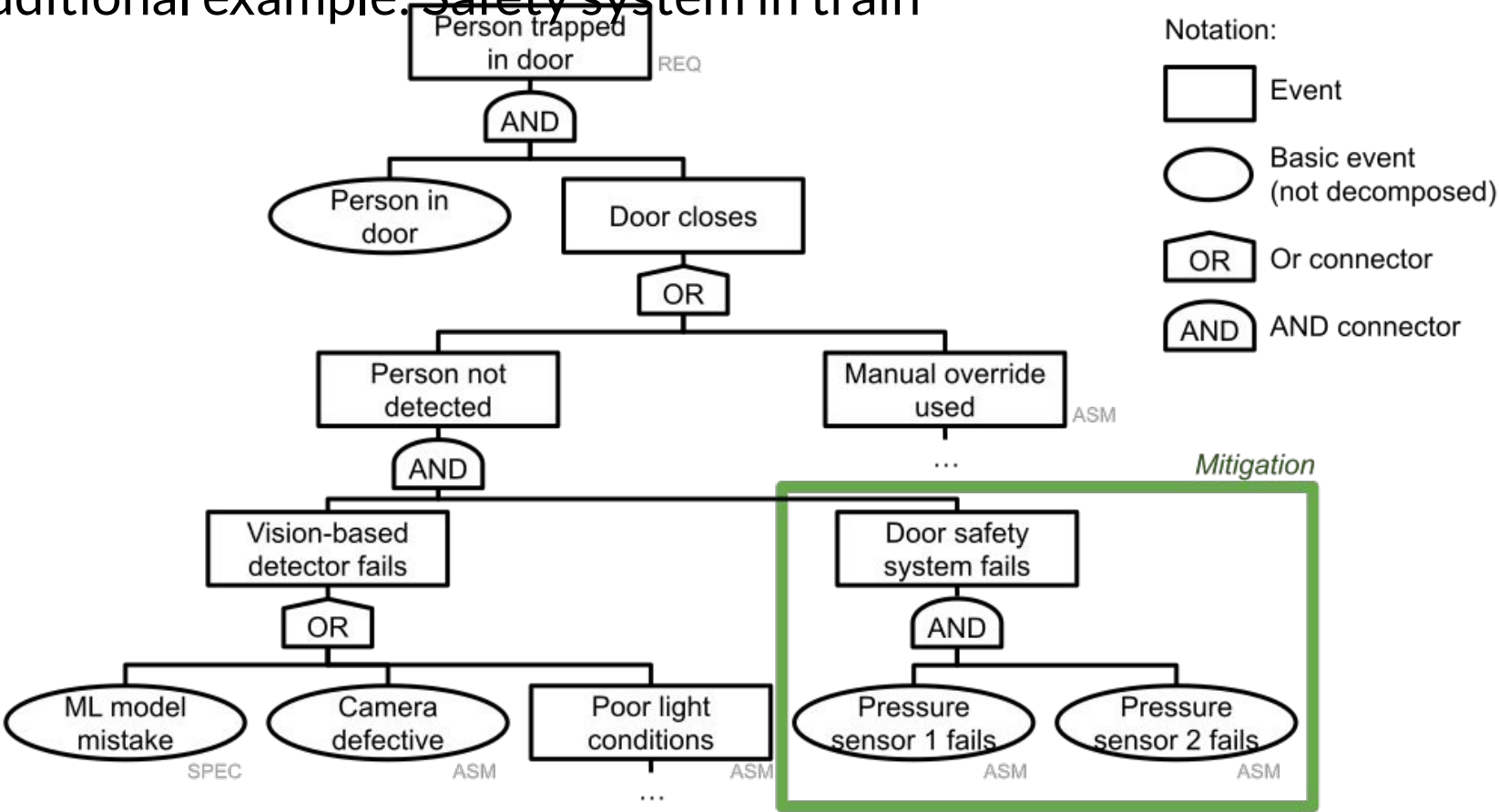
Additional example: Safety system in train



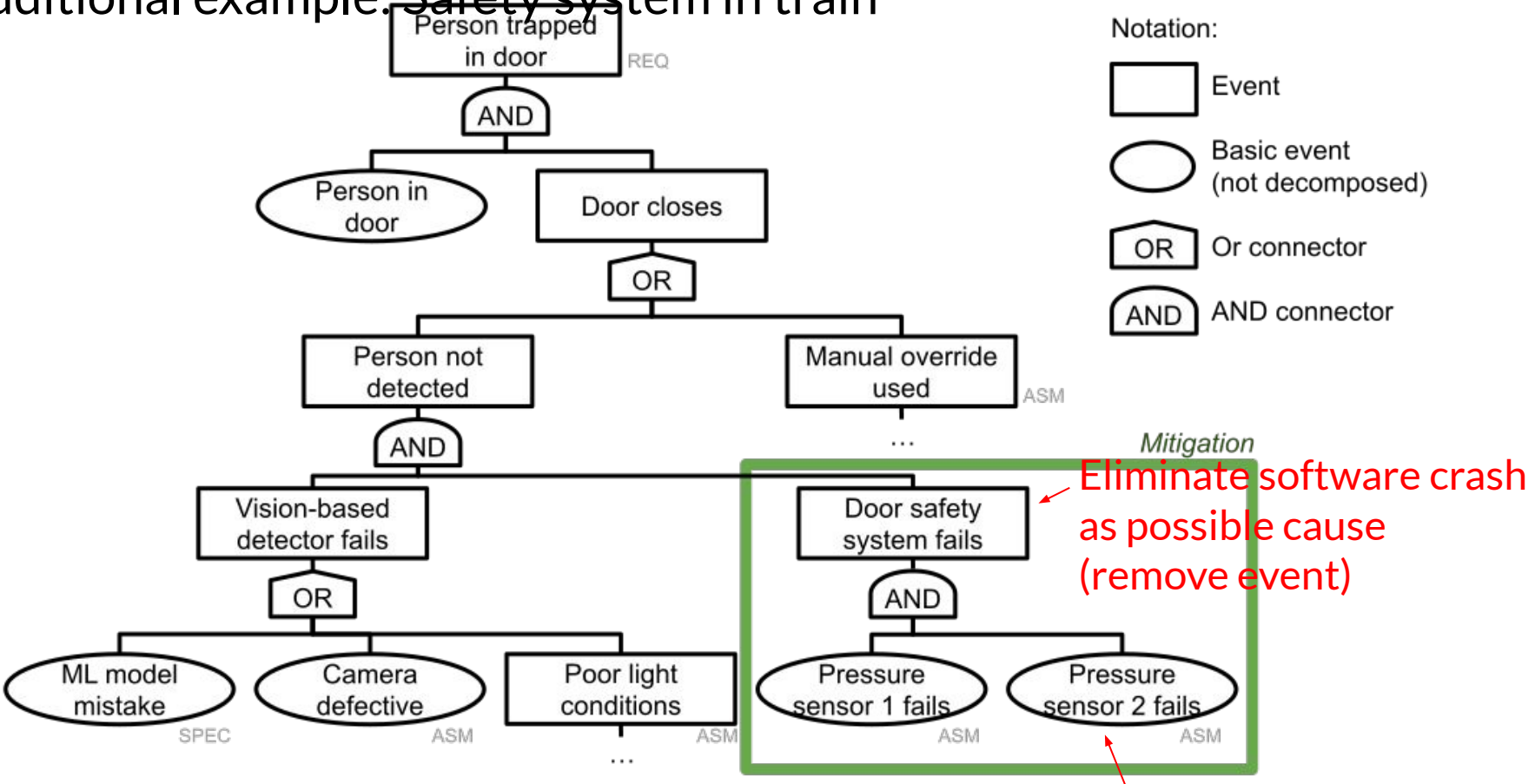
Terrible design, have safe default

Single point of failure?

Additional example: Safety system in train



Additional example: Safety system in train



Breakout: Fault Tree

Expense management system REQ: The system reimburse the correct total amount of approved claims within 5 days.

As a group,

- (a) draw a small fault tree involving at least 1 non-ML and one ML-related basic event
- (b) introduce a mitigation and highlight it in the diagram

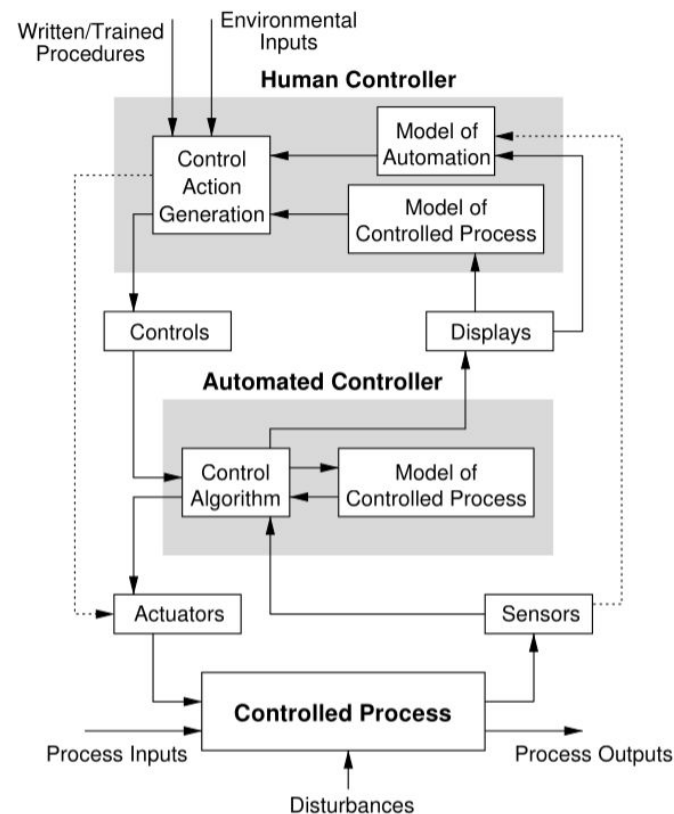
Use pen&paper or any software. As a group, post photo or screenshot to #lecture and tag all group members and @mlipbot. Follow up on @mlipbot comments.

Aside: STPA View

Most losses are caused by complex issues, not just single mistakes

Consider the entire system, control structure, including humans and their training/oversight

Systematically evaluate whether the controls are effective



Zooming Out: General Safety Engineering Strategies

Identify possible hazards from stakeholder goals (STPA, early steps)

Identify possible hazards from component failures (FMEA, HAZOP)

Forward: from cause to hazard

Analyze causes of anticipated/known hazards (FTA)

Backward: from hazard to cause

Analyze effectiveness of control mechanisms for anticipated/known hazards, including non-technical controls (STPA)

Summary

Summary

Accept that a failure is inevitable

- ML components will eventually make mistakes, reasons barely matter
- Environment may evolve over time, violating assumptions

Design strategies for mitigating mistakes

- Human in the loop, undoable actions, guardrails, mistake detection and recovery (monitoring, doer-checker, fail-over), redundancy, containment and isolation

Use risk analysis to identify potential problems (STPA, FTA, ...)

Further readings

- Google PAIR. People + AI Guidebook. 2019, especially chapters “Errors + Graceful Failure” and “Mental Models.”
- Martelaro, Nikolas, Carol J. Smith, and Tamara Zilovic. “Exploring Opportunities in Usable Hazard Analysis Processes for AI Engineering.” In AAAI Spring Symposium Series Workshop on AI Engineering: Creating Scalable, Human-Centered and Robust AI Systems (2022).
- Qi, Yi, Philippa Ryan Conmy, Wei Huang, Xingyu Zhao, and Xiaowei Huang. “A Hierarchical HAZOP-Like Safety Analysis for Learning-Enabled Systems.” In AISafety2022 Workshop at IJCAI2022 (2022).
- Beachum, David Robert. “Methods for assessing the safety of autonomous vehicles.” MSc thesis, 2019.
- Amershi, Saleema, Dan Weld, Mihaela Vorvoreanu, Adam Fourney, Besmira Nushi, Penny Collisson, Jina Suh et al. “Guidelines for human-AI interaction.” In Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems, 2019.
- Shneiderman, Ben. “Bridging the gap between ethics and practice: Guidelines for reliable, safe, and trustworthy Human-Centered AI systems.” ACM Transactions on Interactive Intelligent Systems (TiiS) 10, no. 4 (2020): 1–31.
- Rismani, Shalaleh, Renee Shelby, Andrew Smart, Edgar Jatho, Joshua Kroll, AJung Moon, and Negar Rostamzadeh. “From plane crashes to algorithmic harm: applicability of safety engineering frameworks for responsible ML.” In Proceedings CHI, pp. 1-18. 2023.
- Hong, Yining, Christopher S. Timperley, and Christian Kästner. “From hazard identification to controller design: Proactive and llm-supported safety engineering for ml-powered systems.” In Proc. CAIN, pp. 113-118. IEEE, 2025.
- Leveson, Nancy G. Engineering a safer world: Systems thinking applied to safety. The MIT Press, 2016.
- STPA Handbook <https://psas.scripts.mit.edu/home/materials/>